

Linked List – Complete Summary

Concepts | Diagrams | Operations | Complexities | Key Points | Problems

1. What is a Linked List?

- A linked list is a linear data structure where elements (nodes) are stored in separate memory locations.
- Each node contains data and link(s) (pointer) to other node(s).
- Nodes are not stored contiguously in memory.
- The nodes are connected using pointers, forming a logical sequence.

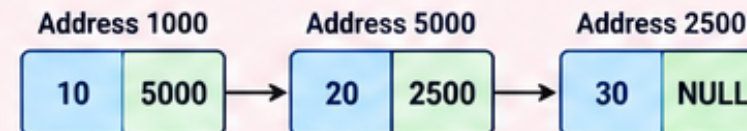
Why do we need Linked List?

- Dynamic size (can grow/shrink easily)
- Efficient insertion and deletion (once node is found)
- No need for contiguous memory allocation
- Useful when size is not known beforehand

2. Node Structure (General Form)



Memory representation (conceptual):



Nodes can be at different memory locations, but pointers connect them logically.

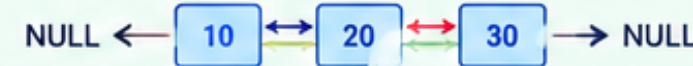
3. Types of Linked Lists

a) Singly Linked List (SLL)



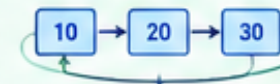
- Each node has **data** and **next**
- Traversal in forward direction only

b) Doubly Linked List (DLL)



- Each node has **data**, **prev** and **next**
- Traversal in both directions

c) Circular Singly Linked List (CSLL)



- Last node points to first node
- No NULL

d) Circular Doubly Linked List (CDLL)



- Last node's next points to first node
- First node's prev points to last node
- No NULL

4. Comparison with Array

Property	Array	Linked List
Memory allocation	Contiguous	Non-contiguous
Size	Fixed (or needs resizing)	Dynamic
Access by index	O(1)	O(n)
Insertion (at beginning)	O(n)	O(1)
Deletion (at beginning)	O(n)	O(1)
Insertion (after known node)	O(n) (need shift)	O(1)
Deletion (after known node)	O(n) (need shift)	O(1)
Extra memory	No	Yes (pointers)
Cache locality	Good	Poorer (scattered)

5. Basic Terminology

- Node:** Basic element containing data and link(s)
- Head:** Pointer to the first node
- Tail:** Last node (optional pointer)
- Next:** Pointer to next node
- Prev:** Pointer to previous node (in DLL)
- NULL:** Indicates end of list (in SLL and DLL)
- Length/Size:** Total number of nodes
- Traversal:** Visiting all nodes
- Insertion:** Adding a new node
- Deletion:** Removing a node
- Search:** Finding a node with given value

6. Common Operations (Overview)

Operation	Description	SLL	DLL	CSLL	CDLL
Traversal	Visit all nodes	O(n)	O(n)	O(n)	O(n)
Insertion at beginning	Add new first node	O(1)	O(1)	O(1)	O(1)
Insertion at end	Add new last node	O(1)	O(1)	O(1)	O(1)
Insertion at position	Add at given index	O(n)	O(n)	O(n)	O(n)
Deletion from beginning	Remove first node	O(1)	O(1)	O(1)	O(1)
Deletion from end	Remove last node	O(n)	O(1)	O(1)	O(1)
Deletion from position	Remove at given index	O(n)	O(n)	O(n)	O(n)
Search	Find a value	O(n)	O(n)	O(n)	O(n)
Length	Count nodes	O(n)	O(n)	O(n)	O(n)
Reverse	Reverse the list	O(n)	O(n)	O(n)	O(n)

7. Pointer Updates – Key Idea

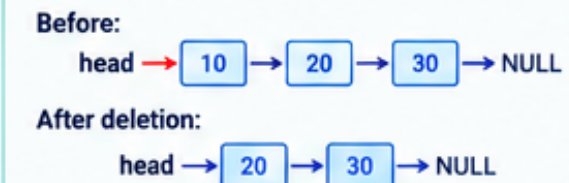
Insertion at beginning (SLL)



Pointer updates: (minimum = 2)

- new_node.next = head
- head = new_node

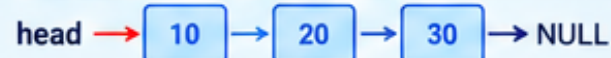
Deletion from beginning (SLL)



Pointer updates: (minimum = 1)

- head = head.next (and free/delete old node)

8. Singly Linked List (SLL)



Characteristics

- Each node has data and next pointer
- Traversal only in forward direction
- Last node's next is NULL

Advantages

- Simple structure
- Less memory per node

Disadvantages

- No backward traversal
- Deletion of last node needs traversal

9. Doubly Linked List (DLL)



Characteristics

- Each node has data, prev and next
- Traversal in both directions
- First node's prev is NULL
- Last node's next is NULL

Advantages

- Easy insertion and deletion at both ends
- Backward traversal possible
- Delete a node without traversal (if node given)

Disadvantages

- Extra memory for prev pointer
- Slower than SLL implementation

10. Circular Singly Linked List (CSLL)



Characteristics

- Each node has data and next
- Last node's next points to first node
- No NULL in the list

Advantages

- Useful for circular processing (e.g., round robin)
- Can start traversal from any node

Disadvantages

- Need special condition to stop (no NULL)
- Slightly more complex implementation

11. Circular Doubly Linked List (CDLL)



Characteristics

- Each node has data, prev and next
- Last node's next points to first node
- First node's prev points to last node
- No NULL in the list

Advantages

- Traversal in both directions
- Useful in circular applications with backward access

Disadvantages

- Extra memory for prev pointer
- More complex implementation

12. Standard Edge Cases to Handle

- Empty list (head == NULL)
- Single node list
- Insert/Delete at beginning when list is empty
- Insert/Delete at end
- Insert/Delete at given position (position 0 or invalid)
- For circular list: stopping condition in traversal
- For doubly/circular list: updating prev pointers correctly

13. Common Mistakes

- Forgetting to update head
- Not handling empty list
- Losing reference to remaining list
- Incorrect order of pointer updates
- Dereferencing NULL
- Not updating prev in doubly linked list
- Infinite loop in circular linked list traversal
- Memory leak in C (not freeing deleted node)

14. Key Takeaways

- A linked list is a collection of nodes connected using pointers.
- Use **head** to represent the list, use **temp** to traverse.
- Insertion/deletion are O(1) if the node/location is already known.
- Otherwise, traversal is needed → O(n).
- DLL allows bidirectional traversal.
- Circular lists have no NULL, so special stopping condition is needed.
- Pointer updates must be done carefully (order matters).
- Always handle edge cases.

15. Important Problems (After Basics)

- Length of linked list
- Search an element
- Reverse linked list
- Find middle node (slow-fast pointers)
- Find nth node from end
- Detect cycle (Floyd's algorithm)
- Find start of cycle
- Check if linked list is palindrome
- Merge two sorted linked lists
- Intersection point of two linked lists